



ROYAUME DU MAROC

المملكة المغربية

Ministère de l'Enseignement Supérieur,
de la Formation des Cadres et de la Recherche Scientifique

CONCOURS NATIONAL COMMUN

d'Admission dans les Établissements de Formation
d'Ingénieurs et Établissements Assimilés

Édition 2021 - 2015

ÉPREUVE D'INFORMATIQUE

Partie : Bases de données

Préparation CNC : Informatique

Prof Youness : 06 78 26 25 20

Filières : **MP/PSI/TSI**

Durée **2** heures

Partie II : Bases de données et langage SQL

Dans le but de suivre l'évolution de la pandémie du Covid-19 dans chaque pays, on propose d'utiliser une base de données composée de 3 tables : **Pays**, **Villes** et **Quotidien**.

a- Structure de la table 'Pays' :

La table '**Pays**' est composée de trois champs : Le champ '**codPays**' de type texte. Ce champ représente la clé primaire de la table '**Pays**'. Le champ '**nom**' de type texte, contient le nom de chaque pays.

Exemples :

codPays	nom
DEU	Allemagne
FRA	France
ESP	Espagne
ARN	Suède
ITA	Italie
...	...

b- Structure de la table 'Villes' :

La table '**Villes**' est composée de 4 champs : Le champ '**codVille**' de type texte. Il représente la clé primaire de la table '**Ville**'. Le champ '**nom**' de type texte, il contient le nom de chaque ville. Le champ '**population**' de type entier, il contient le nombre de personnes dans chaque ville. Le champ '**codP**' de type texte, il contient le code du pays de chaque ville.

Exemples :

codVille	nom	population	codP
CIA	Rome	2 844 395	ITA
NAP	Napoli	954 244	ITA
PAR	Paris	2 148 271	FRA
TXL	Berlin	3 748 148	DEU
LYS	Lyon	1 669 730	FRA
...	...	...	...

c- Structure de la table 'Quotidien' :

La table '**Quotidien**' est composée de 5 champs : Le champ '**jour**' de type date, il contient la date de chaque jour. Le champ '**infectés**' de type entier, il contient le nombre de nouveaux cas de personnes infectées par le virus, chaque jour dans chaque ville. Le champ '**rétablis**' de type entier, il contient le nombre de nouveaux cas de personnes guéries du virus, chaque jour dans chaque ville. Le champ '**décédés**' de type entier, il contient le nombre de nouveaux cas de personnes décédées à cause du virus, chaque jour dans chaque ville. Le champ '**codV**' de type texte, il contient les codes des villes.

Exemples :

jour	infectés	rétablis	décédés	codeV
2020-07-15	4653	3458	172	CIA
2020-07-15	6074	4892	493	TXL
2020-07-15	1762	2059	251	LYS
2020-07-16	3721	5973	85	CIA
...	...			...

Q.1- Le jour du '17/09/2020', la ville de code 'TXL' a enregistré **2513** nouveaux cas de personnes infectées, **2847** nouveaux cas de personnes rétablies, et **65** nouveaux cas de personnes décédées.

Écrire, en langage **SQL**, la requête qui ajoute cet enregistrement dans la table 'Quotidien'.

Q.2- Écrire, en langage **SQL**, la requête qui donne pour résultat, le compte des villes qui ont enregistré des nombres de nouveaux cas de personnes infectées, composés d'au moins **5** chiffres, et ce le jour du '27/08/2020'.

Q.3- Écrire, en langage **SQL**, la requête qui donne pour résultat, le nombre de nouveaux cas de personnes infectées, le nombre nouveaux cas de personnes rétablies et le nombre de nouveaux cas de personnes décédées, chaque jour du mois de **Septembre 2020**, dans toutes les villes du pays 'Italie'.

Q.4- Écrire, en algèbre relationnelle, la requête de la question **Q.3**

Q.5- *Le taux d'incidence **TI** pour une ville est égal au nombre de nouveaux cas de personnes infectées dans cette ville, pendant une période donnée, divisé par la population de cette ville. Le **TI** s'exprime en nombre de personnes pour 100000 personnes.*

Écrire, en langage **SQL**, la requête qui donne pour résultat, les codes des pays, les noms des villes et le **TI** de chaque ville, pendant l'année **2020**, tels que **TI** est compris entre **500** et **1000**, triés dans l'ordre croissant du **TI**.

Q.6- *Le taux de guérison **TG** pour un pays est égal au nombre total de personnes rétablies dans ce pays, divisé par le nombre total de personnes infectées dans ce même pays. Le **TG** s'exprime en pourcentage.*

Écrire, en langage **SQL**, la requête qui donne pour résultat, les noms des pays et le **TG** pour chaque pays, tels que **TG > 70%**, triés dans l'ordre décroissant du **TG**.

Q.7- *Le taux de mortalité **TM** pour une ville est égal au nombre total de personnes décédées dans cette ville, divisé par le nombre total de personnes infectées dans cette même ville. Le **TM** s'exprime en pourcentage.*

Écrire, en langage **SQL**, la requête qui donne pour résultat, les noms des pays, le plus petit **TM**, le plus grand **TM** et la moyenne des **TM** des villes dans chaque pays, triés dans l'ordre croissant du nombre de villes dans chaque pays.

Partie II : Bases de données et langage SQL

Dans le but de suivre l'évolution de la pandémie du Covid-19 dans chaque pays, on propose d'utiliser une base de données composée de 3 tables : Pays, Villes et Quotidien.

a- Structure de la table 'Pays' :

La table 'Pays' est composée de ~~trois~~ *deux* champs : Le champ 'codPays' de type texte. Ce champ représente la clé primaire de la table 'Pays'. Le champ 'nom' de type texte, contient le nom de chaque pays.

Exemples :

codPays	nom
DEU	Allemagne
FRA	France
ESP	Espagne
ARN	Suède
ITA	Italie
...	...

b- Structure de la table 'Villes' :

La table 'Villes' est composée de 4 champs : Le champ 'codVille' de type texte. Il représente la clé primaire de la table 'Ville'. Le champ 'nom' de type texte, il contient le nom de chaque ville. Le champ 'population' de type entier, il contient le nombre de personnes dans chaque ville. Le champ 'codP' de type texte, il contient le code du pays de chaque ville.

Exemples :

codVille	nom	population	codP
CIA	Rome	2 844 395	ITA
NAP	Napoli	954 244	ITA
PAR	Paris	2 148 271	FRA
TXL	Berlin	3 748 148	DEU
LYS	Lyon	1 669 730	FRA
...	...	...	...

c- Structure de la table 'Quotidien' :

La table 'Quotidien' est composée de 5 champs : Le champ 'jour' de type date, il contient la date de chaque jour. Le champ 'infectés' de type entier, il contient le nombre de nouveaux cas de personnes infectées par le virus, chaque jour dans chaque ville. Le champ 'rétablis' de type entier, il contient le nombre de nouveaux cas de personnes guéries du virus, chaque jour dans chaque ville. Le champ 'décédés' de type entier, il contient le nombre de nouveaux cas de personnes décédées à cause du virus, chaque jour dans chaque ville. Le champ 'codV' de type texte, il contient les codes des villes.

Exemples :

jour	infectés	rétablis	décédés	codeV
2020-07-15	4653	3458	172	CIA
2020-07-15	6074	4892	493	TXL
2020-07-15	1762	2059	651	LYS
2020-07-16	3721	5973	85	CIA
-	-			-

Q.1- Le jour du '17/09/2020', la ville de code 'TXL' a enregistré 2513 nouveaux cas de personnes infectées, 2847 nouveaux cas de personnes rétablies, et 65 nouveaux cas de personnes décédées.

Écrire, en langage SQL, la requête qui ajoute cet enregistrement dans la table 'Quotidien'.

Q.2- Écrire, en langage SQL, la requête qui donne pour résultat, le compte des villes qui ont enregistré des nombres de nouveaux cas de personnes infectées, composés d'au moins 5 chiffres, et ce le jour du '27/08/2020'.

Q.3- Écrire, en langage SQL, la requête qui donne pour résultat, le nombre de nouveaux cas de personnes infectées, le nombre nouveaux cas de personnes rétablies et le nombre de nouveaux cas de personnes décédées, chaque jour du mois de Septembre 2020, dans toutes les villes du pays 'Italie'.

Q.4- Écrire, en algèbre relationnelle, la requête de la question Q.3

Q.5- Le taux d'incidence TI pour une ville est égal au nombre de nouveaux cas de personnes infectées dans cette ville, pendant une période donnée, divisé par la population de cette ville. Le TI s'exprime en nombre de personnes pour 100000 personnes.

Écrire, en langage SQL, la requête qui donne pour résultat, les codes des pays, les noms des villes et le TI de chaque ville, pendant l'année 2020, tels que TI est compris entre 500 et 1000, triés dans l'ordre croissant du TI .

Q.6- Le taux de guérison TG pour un pays est égal au nombre total de personnes rétablies dans ce pays, divisé par le nombre total de personnes infectées dans ce même pays. Le TG s'exprime en pourcentage.

Écrire, en langage SQL, la requête qui donne pour résultat, les noms des pays et le TG pour chaque pays, tels que $TG > 70\%$, triés dans l'ordre décroissant du TG .

Q.7- Le taux de mortalité TM pour une ville est égal au nombre total de personnes décédées dans cette ville, divisé par le nombre total de personnes infectées dans cette même ville. Le TM s'exprime en pourcentage.

Écrire, en langage SQL, la requête qui donne pour résultat, les noms des pays, le plus petit TM , le plus grand TM et la moyenne des TM des villes dans chaque pays, triés dans l'ordre croissant du nombre de villes dans chaque pays.

Partie II : Base de données et langage SQL

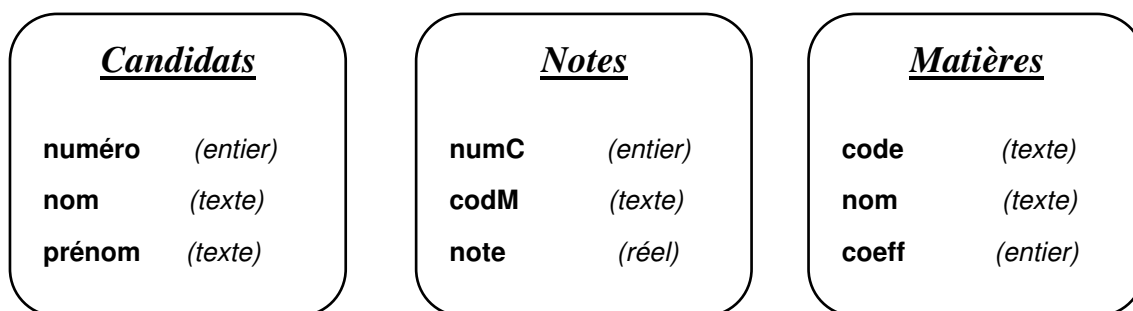
Classement des candidats

Une grande école d'ingénieurs organise un concours au profit des étudiants (candidats), qui veulent y accéder pour suivre leurs études supérieures. Le concours est composé de plusieurs épreuves : une seule épreuve par matière.

Après le passage des épreuves, une note totale est calculée pour chaque candidat. Ensuite les candidats sont classés dans l'ordre décroissant de la note totale.

*NB : Une note inférieure strictement à **5.0** est une note éliminatoire du classement final. Si un candidat possède au moins une note éliminatoire, alors ce candidat sera exclu du classement final.*

L'école d'ingénieurs utilise une base de données relationnelle composée de trois tables :



La table '**Candidats**' contient les numéros, les noms et les prénoms des candidats. Le champ **numéro** est la clé primaire dans cette table.

Exemples :

numéro	nom	prénom
416	Jarfaoui	Hicham
70	Hilal	Samira
162	Senhaji	Amal
23	Bakouri	Ahmed
...	...	...

La table '**Matières**' contient les codes, les noms et les coefficients correspondants à chaque matière. Le champ **code** est la clé primaire dans cette table.

Exemples :

code	nom	coeff
M	Mathématiques	14
P	Physique	10
SI	Sciences de l'ingénieur	6
Ch	Chimie	3
...	...	...

La table '**Notes**' contient, pour chaque candidat, la note correspondante à chaque matière. Les champs **numC** et **codM** sont deux clés étrangères, qui font respectivement référence, aux champs **numéro** et **code** des tables 'Candidats' et 'Matières'.

Exemples :

numC	codM	note
70	SI	14,50
162	SI	17,00
416	SI	12,25
70	M	16,00
162	M	13,50
416	M	08,35
70	P	04,75
162	P	11,05
416	P	10,10
...	...	...

Q.1 – Déterminer la clé primaire de la table 'Notes', et justifier votre réponse.

Q.2 – Écrire, en algèbre relationnelle, une requête qui donne les numéros, les noms et prénoms de tous les candidats, qui ne possèdent pas de note éliminatoire, en matière de code '**M**'.

Q.3 – Écrire la requête précédente (**Q.2**) en langage SQL.

Q.4 – Écrire, en langage SQL, une requête qui donne les noms des matières, la note maximale et la note minimale de chaque matière, triés dans l'ordre décroissant de la moyenne des notes de chaque matière.

Q.5 – Écrire, en langage SQL, une requête qui donne le compte des candidats qui sont exclus du classement final.

Q.6– Pour les candidats non exclus du classement final, la **note totale** de chaque candidat est calculée par la formule suivante :

$$note\ totale = \sum (note * coefficient)$$

La note d'une matière est multipliée par le coefficient correspondant à cette matière.

Écrire, en langage SQL, une requête qui donne le numéro, le nom, le prénom et la note totale de chaque candidat non exclu, ayant la note totale supérieure strictement à **1000.0**, triés dans l'ordre décroissant de la note totale.

Partie II : Base de données et langage SQL

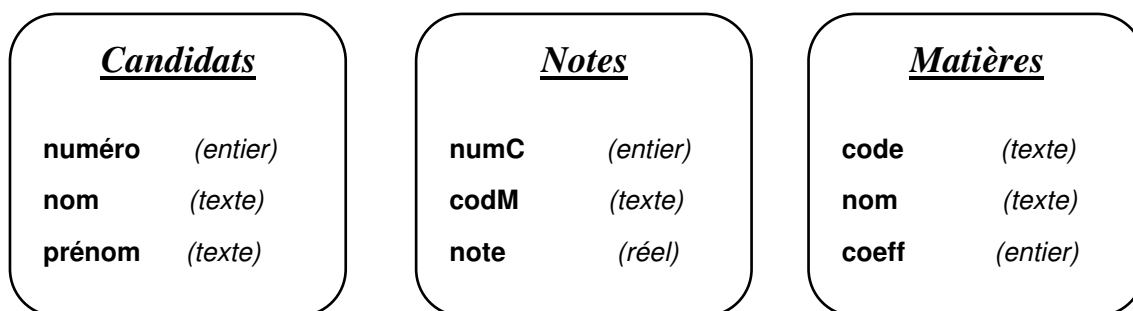
Classement des candidats

Une grande école d'ingénieurs organise un concours au profit des étudiants (candidats), qui veulent y accéder pour suivre leurs études supérieures. Le concours est composé de plusieurs épreuves : une seule épreuve par matière.

Après le passage des épreuves, une note totale est calculée pour chaque candidat. Ensuite les candidats sont classés dans l'ordre décroissant de la note totale.

*NB : Une note inférieure à **5.0** est une note éliminatoire du classement final. Si un candidat possède au moins une note éliminatoire, alors ce candidat sera exclu du classement final.*

L'école d'ingénieurs utilise une base de données relationnelle composée de trois tables :



La table '**Candidats**' contient les numéros, les noms et les prénoms des candidats. Le champ **numéro** est la clé primaire dans cette table.

Exemples :

numéro	nom	prénom
416	Jarfaoui	Hicham
70	Hilal	Samira
162	Senhaji	Amal
23	Bakouri	Ahmed
...	...	...

La table '**Matières**' contient les codes, les noms et les coefficients des matières. Le champ **code** est la clé primaire dans cette table.

Exemples :

code	nom	coeff
M	Mathématiques	14
P	Physique	10
SI	Sciences de l'ingénieur	6
Ch	Chimie	3
...	...	...

La table '**Notes**' contient, pour chaque candidat, la note correspondante à chaque matière. Les champs **numC** et **codM** sont deux clés étrangères, qui font respectivement référence, aux champs **numéro** et **code** des tables 'Candidats' et 'Matières'.

Exemples :

numC	codM	note
70	SI	14,50
162	SI	17,00
416	SI	12,25
70	M	16,00
162	M	13,50
416	M	08,35
70	P	04,75
162	P	11,05
416	P	10,10
...	...	...

Q.1 – Déterminer la clé primaire de la table 'Notes', et justifier votre réponse.

Q.2 – Écrire, en algèbre relationnelle, une requête qui donne les numéros, les noms et prénoms de tous les candidats, qui ne possèdent pas de note éliminatoire, en matière de code '**M**'.

Q.3 – Écrire la requête précédente (**Q.2**) en langage SQL.

Q.4 – Écrire, en langage SQL, une requête qui donne les noms des matières, la note maximale et la note minimale de chaque matière, triés dans l'ordre décroissant de la moyenne des notes de chaque matière.

Q.5 – Écrire, en langage SQL, une requête qui donne le compte des candidats qui sont exclus du classement final.

Q.6– Pour les candidats non exclus du classement final, la **note totale** de chaque candidat est calculée par la formule suivante :

$$note\ totale = \sum (note * coefficient)$$

La note d'une matière est multipliée par le coefficient correspondant à cette matière.

Écrire, en langage SQL, une requête qui donne le numéro, le nom, le prénom et la note totale de chaque candidat non exclu, ayant la note totale supérieure strictement à **1000.0**, triés dans l'ordre décroissant de la note totale.

Partie II : Base de données et langage SQL

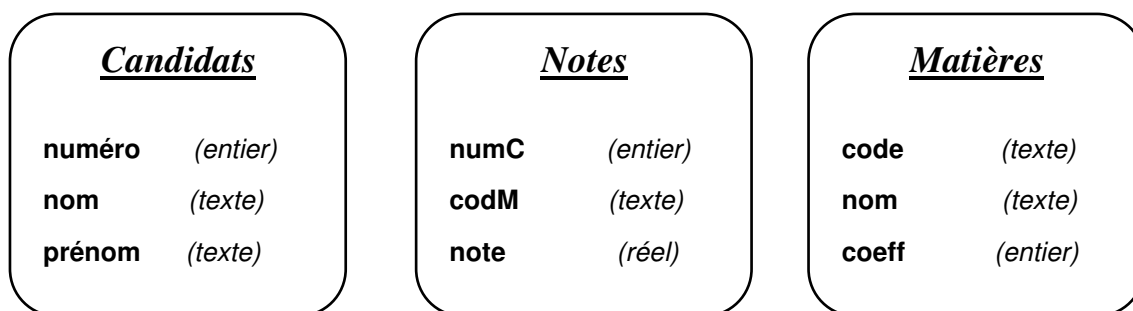
Classement des candidats

Une grande école d'ingénieurs organise un concours au profit des étudiants (candidats), qui veulent y accéder pour suivre leurs études supérieures. Le concours est composé de plusieurs épreuves : une seule épreuve par matière.

Après le passage des épreuves, une note totale est calculée pour chaque candidat. Ensuite les candidats sont classés dans l'ordre décroissant de la note totale.

*NB : Une note inférieure à **5.0** est une note éliminatoire du classement final. Si un candidat possède au moins une note éliminatoire, alors ce candidat sera exclu du classement final.*

L'école d'ingénieurs utilise une base de données relationnelle composée de trois tables :



La table '**Candidats**' contient les numéros, les noms et les prénoms des candidats. Le champ **numéro** est la clé primaire dans cette table.

Exemples :

numéro	nom	prénom
416	Jarfaoui	Hicham
70	Hilal	Samira
162	Senhaji	Amal
23	Bakouri	Ahmed
...	...	...

La table '**Matières**' contient les codes, les noms et les coefficients des matières. Le champ **code** est la clé primaire dans cette table.

Exemples :

code	nom	coeff
M	Mathématiques	14
P	Physique	10
SI	Sciences de l'ingénieur	6
Ch	Chimie	3
...	...	...

La table '**Notes**' contient, pour chaque candidat, la note correspondante à chaque matière. Les champs **numC** et **codM** sont deux clés étrangères, qui font respectivement référence, aux champs **numéro** et **code** des tables 'Candidats' et 'Matières'.

Exemples :

numC	codM	note
70	SI	14,50
162	SI	17,00
416	SI	12,25
70	M	16,00
162	M	13,50
416	M	08,35
70	P	04,75
162	P	11,05
416	P	10,10
...	...	...

Q.1 – Déterminer la clé primaire de la table 'Notes', et justifier votre réponse.

Q.2 – Écrire, en algèbre relationnelle, une requête qui donne les numéros, les noms et prénoms de tous les candidats, qui ne possèdent pas de note éliminatoire, en matière de code '**M**'.

Q.3 – Écrire la requête précédente (**Q.2**) en langage SQL.

Q.4 – Écrire, en langage SQL, une requête qui donne les noms des matières, la note maximale et la note minimale de chaque matière, triés dans l'ordre décroissant de la moyenne des notes de chaque matière.

Q.5 – Écrire, en langage SQL, une requête qui donne le compte des candidats qui sont exclus du classement final.

Q.6– Pour les candidats non exclus du classement final, la **note totale** de chaque candidat est calculée par la formule suivante :

$$note\ totale = \sum (note * coefficient)$$

La note d'une matière est multipliée par le coefficient correspondant à cette matière.

Écrire, en langage SQL, une requête qui donne le numéro, le nom, le prénom et la note totale de chaque candidat non exclu, ayant la note totale supérieure strictement à **1000.0**, triés dans l'ordre décroissant de la note totale.

Partie I :

Bases de données et langage SQL

Dans les réseaux sociaux, on peut créer des groupes, et d'inviter des personnes à ces groupes. Dans le but de faire des statistiques sur les groupes et leurs membres, nous avons créé une base de données relationnelle composée de trois tables : **Personnes**, **Membres** et **Groupes**.

A- Structure de la table 'Personnes' :

La table 'Personnes' contient les personnes qui ont créé des groupes, et les personnes qui ont accepté l'invitation pour se joindre aux groupes, et devenir membres de ces groupes.

- ✚ Le champ **code** de type entier, contient un entier positif unique pour chaque personne ;
- ✚ Le champ **nom** de type texte, contient le nom de chaque personne ;
- ✚ Le champ **prenom** de type texte, contient le prénom de chaque personne ;
- ✚ Le champ **dateNaiss** de type date, contient la date de naissance de chaque personne.

Exemples d'enregistrements dans la table 'Personnes' :

code	nom	prenom	dateNaiss
1	Boudara	Mohamed	1996-10-15
2	Mernissi	Salima	2000-03-29
3	Sabri	Ahmed	2001-02-20
4	Oubaidi	Mouniya	1990-05-25
5	Jamali	Ahmed	1997-09-08
6	Bekkali	Hamza	1999-11-05
7	Chafi	Asmaa	2002-03-13
8	Chahbouni	Soukaina	2000-01-20
...	...	...	...

B- Structure de la table 'Groupes' :

La table 'Groupes' contient des informations relatives à chaque groupe.

- ✚ Le champ **id** de type entier, contient les identifiants des groupes. L'identifiant de chaque groupe est unique ;
- ✚ Le champ **nom** de type texte, contient les noms des groupes ;
- ✚ Le champ **description** de type texte, contient un texte qui précise les objectifs de chaque groupe ;
- ✚ Le champ **codeCr**, de type date, contient les codes des personnes ayant créé les groupes ;
- ✚ Le champ **dateCr**, de type date, contient la date de création de chaque groupe.

Exemples d'enregistrements dans la table 'Groupes' :

id	nom	description	codeCr	dateCr
253	Amis	Les amis les plus proches	2	2017-07-09
471	Fans	Discussion à propos de divers sujets	6	2016-09-14
96	Famille	Les membres de la grande famille	25	2016-10-10
	...	...	...	...

C- Structure de la table 'Membres' :

La table 'Membres' est composée de trois champs :

- ✚ Le champ **code** de type entier, contient les codes des personnes qui ont accepté l'invitation pour devenir membres aux groupes ;
- ✚ Le champ **id** de type entier, contient les identifiants des groupes ;
- ✚ Le champ **dateM** de type date, contient les dates aux quelles chaque personne a rejoint un groupe.

Exemples d'enregistrements dans la table 'Membres' :

code	id	dateM
1	253	2017-09-18
3	471	2018-01-30
4	253	2016-09-25
5	253	2018-01-22
7	253	2017-07-29
8	471	2016-11-10
...	...	...

I. 1- Déterminer les clés primaires et les clés étrangères dans les tables de cette base de données. Justifier votre réponse.

Écrire, en langage **SQL**, les requêtes qui donnent les résultats suivants :

I. 2- Les personnes qui sont nées le mois **6**, triées dans l'ordre croissant des âges (du moins âgée au plus âgée).

I. 3- Les identifiants des groupes, les dates de création des groupes et les noms et prénoms des personnes ayant crée ces groupes.

I. 4- Écrire la requête **I. 3** en algèbre relationnelle.

I. 5- Les noms et les descriptions des groupes qui contiennent au moins **1000** personnes, triés dans l'ordre décroissant des nombres de personnes.

I. 6- Les codes, les noms et les prénoms de toutes les personnes (créateur et membres) qui appartiennent au groupe ayant le nom : '**Sport**'.

I. 7- Supprimer les personnes qui n'appartiennent à aucun groupe.

I. 8- Modifier dans la table 'Membres', pour que tous les membres du groupe ayant le nom '**Sport**' deviennent membres du groupe ayant le nom '**Voyage**'.

Épreuve d'Informatique – Session 2019 – Filière PSI

Base de données et langage SQL

Réseau routier pondéré :

On associe à chaque route du réseau routier une valeur appelée : **poids**. Le poids de chaque route est la longueur de cette route, exprimée en km (kilomètre). Ainsi, le réseau routier est appelé : *réseau routier pondéré*.

Exemple :

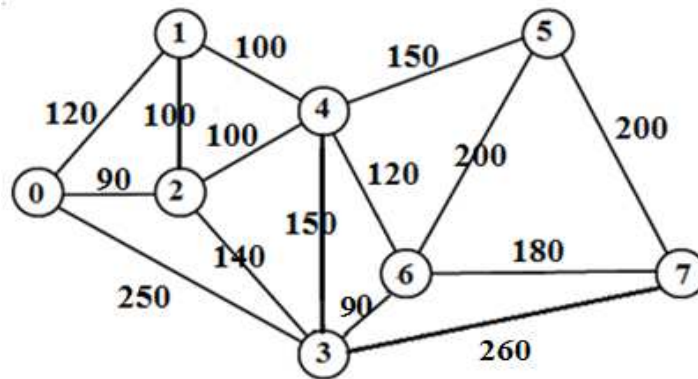


Figure 3 : Réseau routier pondéré par les longueurs des routes en km

Le coût d'un chemin :

Dans un réseau routier pondéré, le **coût d'un chemin** est égale à la somme des poids des routes qui composent ce chemin.

Exemples :

Dans le réseau routier pondéré de la figure 3 :

- Le coût du chemin **(1, 0, 3, 6)** est : **460** = 120+250+90
- Le coût du chemin **(4, 6, 5, 7, 3)** est : **780** = 120+200+200+260
- Le coût du chemin **(0, 1, 2, 3, 4, 5, 6, 7)** est : **1040** = 120+100+140+150+150+200+180

Dans cette partie, on propose de manipuler le réseau routier pondéré par une base de données composée de deux tables : la table : **Villes** et la table : **Chemins**.

Villes

code	(entier)
nom	(texte)
couleur	(entier)

Chemins

chemin	(texte)
nb_routes	(entier)
cout	(réel)
ville_depart	(entier)
ville_arrivee	(entier)

Structure de la table 'Villes' :

La table **Villes** contient les villes du réseau routier.

- ✚ Le champ **code** contient les entiers : 0, 1, 2, 3, ..., **n-1**, sachant que **n** est le nombre de villes dans le réseau routier. Chaque entier représente une seule ville du réseau routier.
- ✚ Le champ **nom** contient les noms des villes ;
- ✚ Le champ **couleur** contient des entiers strictement positifs, qui représentent les couleurs des villes.

Exemples d'enregistrements dans les tables 'Villes' :

code	nom	couleur
0	Agadir	2
1	Essaouira	1
2	Marrakech	3
3	Ouarzazate	1
4	Safi	2
20	Tanger	7
...	...	...

Structure de la table 'Chemins' :

La table **Chemins** est composée de 5 champs :

- ✚ Le champ **chemin** contient tous les chemins simples qui existent entre les villes du réseau routier. Chaque chemin est représenté par une chaîne de caractères, qui contient les villes du chemin, séparées par le caractère '-' (tiré de 6).

Exemple : La chaîne '1-0-3-6' représente le chemin (1, 0, 3, 6)

- ✚ Le champ **nb_routes** contient le nombre de routes dans chaque chemin ;
- ✚ Le champ **cout** contient les coûts des chemins ;
- ✚ Le champ **ville_depart** contient le code de la ville de départ de chaque chemin ;
- ✚ Le champ **ville_arrivee** contient le code de la ville d'arrivée de chaque chemin.

Exemples d'enregistrements dans la table 'Chemins' :

chemin	nb_routes	cout	ville_depart	ville_arrivee
0-1	1	120	0	1
1-0	1	120	1	0
2-4-1	2	200	2	1
1-4-2	2	200	1	2
1-0-3-6	3	460	1	6
4-6-5-7-3	4	780	4	3
0-1-2-3-4-5-6-7	7	1040	0	7
1-4-5-7-6-3	5	720	1	3
5-7-6-4-1-0-2-3	7	950	5	3
...	...	...	...	...

V. 14- Déterminer les clés primaires et les clés étrangères dans les tables **Villes** et **Chemins**.

Justifier votre réponse.

V. 15- Écrire, en algèbre relationnelle, la requête qui donne le résultat suivant :

Les codes et noms des villes ayant la couleur **1** ou la couleur **2** dans le réseau routier.

Écrire, en langage SQL, les requêtes qui donne les résultats suivants :

V. 16- Le nombre de routes dans le réseau routier.

V. 17- Le nombre chromatique du réseau routier.

V. 18- Les noms des villes et le degré de chaque ville, ayant le degré compris strictement entre **4** et **9**, triés dans l'ordre décroissant des degrés.

V. 19- Modifier les coûts de tous les chemins qui passent par la route "**0-1**", en ajoutant la valeur **50** à chacun de ces chemins.

V. 20- Le plus petit coût des chemins qui passent par toutes les villes du réseau routier.

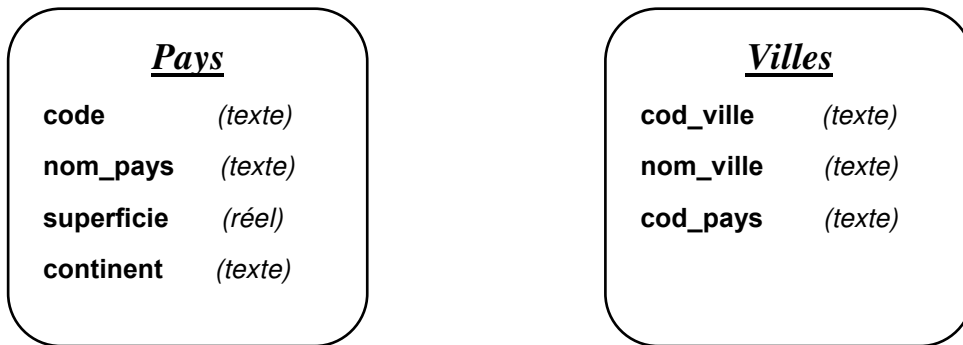
~~~~~ **FIN DE L'ÉPREUVE** ~~~~~

# Épreuve d'Informatique – Session 2019 – Filière TSI

## Partie I :

### *Base de données et Langage SQL*

On considère la base de données relationnelle composée de deux tables : la table **Pays** et la table **Villes**.



#### Structure de la table 'Pays' :

La table '**Pays**' contient des informations sur les pays, elle est composée de **4** champs :

- ✚ Le champ **code** contient un code unique pour chaque pays ;
- ✚ Le champ **nom\_pays** contient le nom de chaque pays ;
- ✚ Le champ **superficie** contient la superficie de chaque pays, exprimée en Km<sup>2</sup> ;
- ✚ Le champ **continent** contient le continent de chaque pays.

#### Exemples d'enregistrements dans la table 'Pays' :

| code | nom_pays  | superficie | continent |
|------|-----------|------------|-----------|
| MAR  | Maroc     | 710 850    | Afrique   |
| DEU  | Allemagne | 357 386    | Europe    |
| ITA  | Italie    | 301 338    | Europe    |
| ARG  | Argentine | 2 780 000  | Amérique  |
| RUS  | Russie    | 1 710 000  | Asie      |
| ...  | ...       | ...        | ...       |

#### Structure de la table 'Villes' :

La table '**Villes**' contient des informations sur les villes des pays, elle est composée de **3** champs :

- ✚ Le champ **cod\_ville** contient un code unique pour chaque ville ;
- ✚ Le champ **nom\_ville** contient les noms des villes ;
- ✚ Le champ **cod\_pays** contient le code du pays de chaque ville.



Exemples d'enregistrements dans la table 'Villes' :

| cod_ville | nom_ville  | cod_pays |
|-----------|------------|----------|
| CIA       | Rome       | ITA      |
| CMN       | Casablanca | MAR      |
| SVO       | Moscou     | RUS      |
| RAK       | Marrakech  | MAR      |
| TXL       | Berlin     | DEU      |
| ...       | ...        | ...      |

**I. 1-** Déterminer les clés primaires et les clés étrangères dans les tables **Pays** et **Villes**.

Justifier votre réponse.

**I. 2-** Écrire, en algèbre relationnelle, la requête qui donne le résultat suivant :

Les noms et les superficies des pays du continent '**Europe**'.

Écrire, en langage SQL, les requêtes qui donnent les résultats suivants :

**I. 3-** Les noms des villes, sans répétition, dans les quelles le caractère '**m**' se trouve à la **2<sup>ème</sup>** et à la **dernière** position. Exemple : **Amsterdam**.

**I. 4-** Les codes et les noms des villes du pays '**Espagne**', triés dans l'ordre alphabétique des noms des villes.

**I. 5-** Les noms des pays, et le nombre de villes dans chaque pays, ayant le nombre de villes compris strictement entre **100** et **1000**, triés dans l'ordre décroissant des nombres de villes.

**I. 6-** Dans certains pays, on peut trouver une ville ayant le même nom que celui de son pays. Donner les noms de ces pays.

**I. 7-** Supprimer toutes les villes du pays '**Belgique**'.

~~~~~

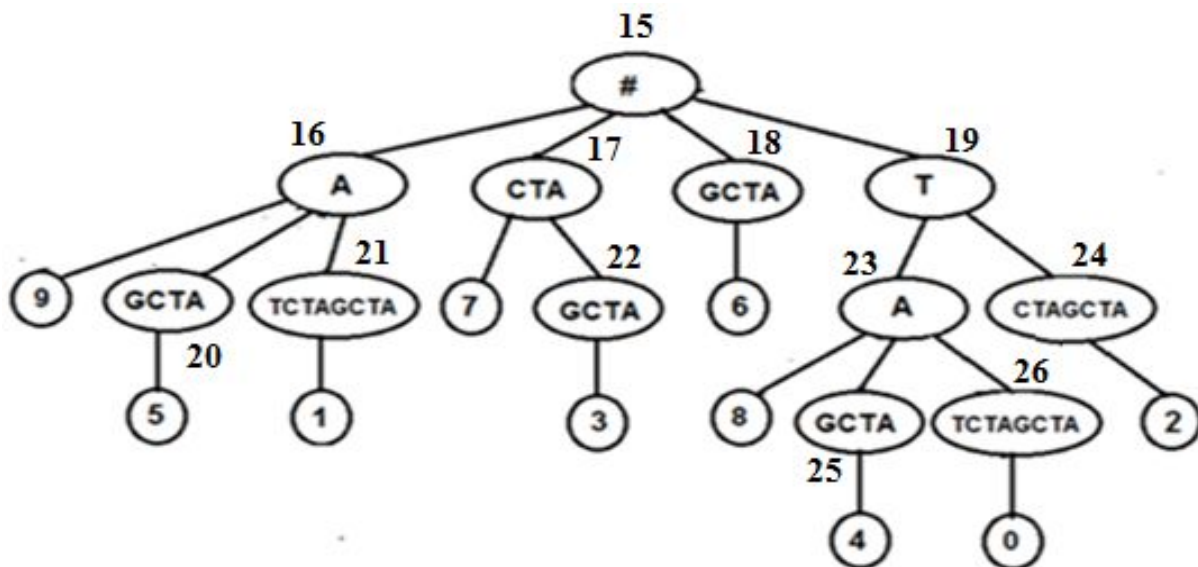
Épreuve d'Informatique – Session 2018 – Filière MP

Base de données d'un arbre des suffixes

Dans cette partie, on utilise une base de données pour stocker un arbre des suffixes d'une chaîne de caractères **S** non vide. Pour identifier les nœuds de l'arbre, on donne un code unique pour chaque nœud :

- Chaque nœud interne est identifié par un entier positif unique supérieur à la taille de **S** ;
- Les feuilles de l'arbre sont les différentes positions **p** dans la chaîne **S** ($0 \leq p < \text{taille de } S$). Chaque feuille de l'arbre est identifiée par l'entier **p** qu'elle contient.

Exemple : Codes des nœuds internes de l'arbre des suffixes de la chaîne **S = 'TATCTAGCTA'**



Dans cet exemple, la taille de **S** est **10**. Chaque feuille est identifiée par un entier compris entre **0** et **9**, et qui correspond à un indice dans **S**. Chaque nœud interne est identifié par un entier positif unique supérieur à **10**. Le choix et la répartition des codes des nœuds sont effectués de façon aléatoire.

La base de données de l'arbre des suffixes est composée de deux tables : **Nœuds** et **Liens**.

a- Structure de la table **Nœuds** :

Nœuds (**code** (entier) , **nom** (texte), **niveau** (entier))

La table **Nœuds** contient les nœuds internes de l'arbre des suffixes, et elle est composée de trois champs :

- ✓ Le champ **code** contient un entier positif unique pour chaque nœud interne ;
- ✓ Le champ **nom** contient le nom du nœud interne ;
- ✓ Le champ **niveau** contient un entier positif qui représente le niveau du nœud interne dans l'arbre.

b- Structure de la table Liens :

Liens (**pere** (entier) , **fil** (entier))

La table **liens** contient toutes les liaisons *père-fils* dans l'arbre, elle est composée de deux champs :

- ✓ Le champ **pere** contient les codes des nœuds pères ;
- ✓ Le champ **fil** contient les codes des nœuds fils ;

c- Exemples d'enregistrements dans les deux tables :

<i>Table : Nœuds</i>		
<i>code</i>	<i>nom</i>	<i>niveau</i>
15	'#'	0
16	'A'	1
17	'CTA'	1
18	'GCTA'	1
19	'T'	1
20	'GCTA'	2
21	'TCTAGCTA'	2
...	...	...
26	'TCTAGCTA'	3

<i>Table : Liens</i>	
<i>pere</i>	<i>fil</i>
15	16
15	17
15	18
15	19
16	9
16	20
16	21
20	5
21	1
...	...

Q. 11 : Déterminer la clé primaire de la table **Nœuds**, et la clé primaire de la table **Liens**.

Justifier votre réponse.

Q. 12 : Écrire une requête SQL, qui renvoie **les noms des nœuds, sans répétition, tels que le motif 'GCTA' est un préfixe ou suffixe des noms de ces nœuds, triés dans l'ordre croissant du champ niveau.**

Q. 13 : Écrire une requête SQL, qui renvoie **la hauteur de l'arbre.**

Exemple : La hauteur de l'arbre exemple est 5.

Q. 14 : Le **degré d'un nœud** est égal au nombre de fils de ce nœud.

Exemple : Le degré de la racine de l'arbre exemple est 4.

Écrire une requête SQL, qui renvoie **les noms et les niveaux des nœuds dont le degré est supérieur à 2, triés dans l'ordre croissant selon le degré.**

Q. 15 : Le **degré d'un arbre** est égal au plus grand des degrés de ses nœuds.

Écrire une requête SQL, qui renvoie **le degré de l'arbre.**

Q. 16 : Écrire une requête SQL, qui renvoie **les feuilles de l'arbre ayant le niveau maximal.**

Q. 17 : Écrire une requête, en algèbre relationnelle, qui renvoie **les feuilles ayant le niveau supérieur à 2.**

Épreuve d'Informatique – Session 2018 – Filière PSI

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

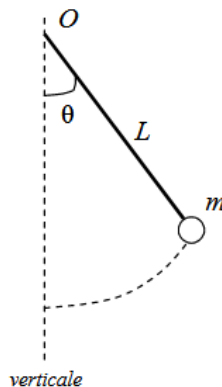
Remarques générales :

- ✓ *L'épreuve se compose de trois parties indépendantes.*
- ✓ *Toutes les instructions et les fonctions demandées seront écrites en Python.*
- ✓ *Les questions non traitées peuvent être admises pour aborder les questions ultérieures.*
- ✓ *Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.*

~ ~ ~ ~ ~

Partie I : Base de données d'un pendule simple

Un pendule simple est constitué d'un solide de masse $\mathbf{m}$, suspendu à une tige rigide de longueur $\mathbf{L}$ et de masse négligeable devant $\mathbf{m}$. Écarté de sa position initiale d'un angle θ et lâché sans vitesse initiale, le pendule effectue des oscillations périodiques autour de sa position d'équilibre définie par : $\theta = 0$.



On considère une base de données, dans laquelle on a enregistré les mesures réalisées sur plusieurs pendules simples. Cette base de données est composée de deux tables : la table **Pendules** et la table **Mesures**.

Pendules

code	(texte)
longueur	(réel)
masse	(réel)
coef_frottement	(réel)

<i>Mesures</i>	
cod_pendule	(texte)
T	(réel)
theta	(réel)

a- Structure de la table Pendules :

La table **Pendules** est composée de **4** champs :

Pendules (code , longueur , masse , coef_frottement)

- ✓ Le champ **code** contient un code unique pour chaque pendule ;
- ✓ Le champ **longueur** contient la longueur de la tige du pendule, exprimée en *mètre* ;
- ✓ Le champ **masse** contient la masse de l'objet suspendu, exprimée en *kilogramme* ;
- ✓ Le champ **coef_frottement** contient le coefficient du frottement. Si la valeur de ce champ est **0**, le pendule est dit : *sans frottement*.

b- Structure de la table mesures :

La table **Mesures** est composée de **3** champs :

Mesures (cod_pendule , T , theta)

- ✓ Le champ **cod_pendule** contient les codes des pendules ;
- ✓ Le champ **T** contient les temps de mesure de l'angle θ , exprimé en *seconde*.

NB : Les mesures de l'angle θ sont prises à un même intervalle de temps, pour chaque pendule.

- ✓ Le champ **theta** contient la mesure de l'angle θ correspondante à chaque instant de **T**, exprimée en *radian*.

c- Exemples de données dans les deux tables :

<i>Table : Pendules</i>			
<i>code</i>	<i>longueur</i>	<i>masse</i>	<i>coef_frottement</i>
P1	0.5	1.0	0.1
P2	0.4	0.5	0.0
P3	1.0	0.5	0.4
P4	0.5	2.0	0.1
P5	1.0	0.5	0.0
...	...	...	...

<i>Table : Mesures</i>		
<i>cod_pendule</i>	<i>T</i>	<i>theta</i>
P1	0	1.047198
P1	0.005	1.046348
P1	0.010	1.045502
P1	0.015	1.043807
P1	0.020	1.042119
...	...	...
P1	10	0.120374
P2	0	1.570796
P2	0.02	1.551176
P2	0.04	1.531556
P2	0.06	1.492324
...	...	...
P2	25	- 0.387401
P3	0	2.356194
P3	0.02	2.350645
P3	0.04	2.345273
P3	...	...

Q.1- Déterminer la clé primaire de la table **Pendules**, et la clé primaire de la table **Mesures**.

Justifier votre réponse.

Q.2- Convertir la requête suivante, en langage SQL :

$$R = \pi_{T, \theta} (\sigma_{\text{cod_pendule} = 'P1'} (\text{Mesures}))$$

Q.3- Écrire une requête SQL, qui donne pour résultat, **les codes, les masses et les longueurs des pendules avec frottement, dont le code ne contient pas le chiffre 5.**

Q.4- Écrire une requête SQL, qui donne pour résultat, **les codes des pendules et les mesures, exprimées en degré, de l'angle θ à l'instant 0, triés dans l'ordre décroissant des mesures de l'angle θ .**

Q.5- Écrire une requête SQL, qui donne pour résultat, **les codes des pendules et le nombre de mesures prises pour chaque pendule, triés dans l'ordre croissant du nombre de mesures.**

Q.6- Pour chaque pendule, les mesures de l'angle θ ont été prises à un même intervalle de temps, appelé : **pas**.

Écrire une requête SQL, qui donne pour résultat, **les codes des pendules sans frottement, ayant pour pas 10^{-2} .**

Q.7- Écrire une requête SQL, qui **crée la nouvelle table 'Pendules_1'**, ayant la structure suivante :

Pendules_1 (**code** (texte) , **theta** (réel) , **Tmax** (réel) , **pas** (réel) , **N** (entier))

Q.8- Écrire une requête SQL, qui **copie, dans la table Pendules_1, le code, l'angle θ initial, le max des T, le pas, et le nombre de mesures prises pour chaque pendule de la table Pendules.**

Exemples de d'enregistrements copiés dans la table **Pendules_1** :

<i>code</i>	<i>theta</i>	<i>Tmax</i>	<i>pas</i>	<i>N</i>
P1	1.047198	10	0.005	2000
P2	1.570796	25	0.02	1250
P3	2.356194	30	0.02	1500
...	...	...	...	...

Épreuve d'Informatique – Session 2018 – Filière TSI

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- ✓ L'épreuve se compose de trois parties indépendantes.
- ✓ Toutes les instructions et les fonctions demandées seront écrites en Python.
- ✓ Les questions non traitées peuvent être admises pour aborder les questions ultérieures.
- ✓ Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~ ~ ~ ~ ~

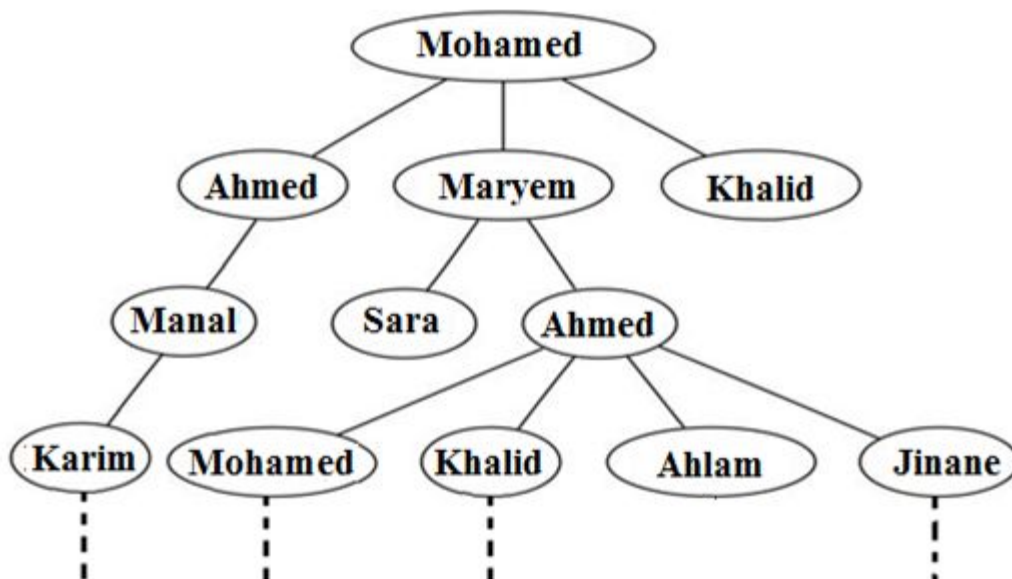
Partie I : Bases de donnée

Arbre généalogique

Dans cette partie, le but est de modéliser une structure qui représente un **arbre généalogique** par une base de données. On commence par représenter l'arbre généalogique sous forme d'un **arbre n-aire descendant** :

En haut de l'arbre se situe l'ancêtre commun à tous les membres de l'arbre généalogique. Et chaque membre de l'arbre est relié vers le bas à chacun de ses enfants. À l'exception de l'ancêtre commun, chaque membre de l'arbre est relié vers le haut à une unique personne : son parent.

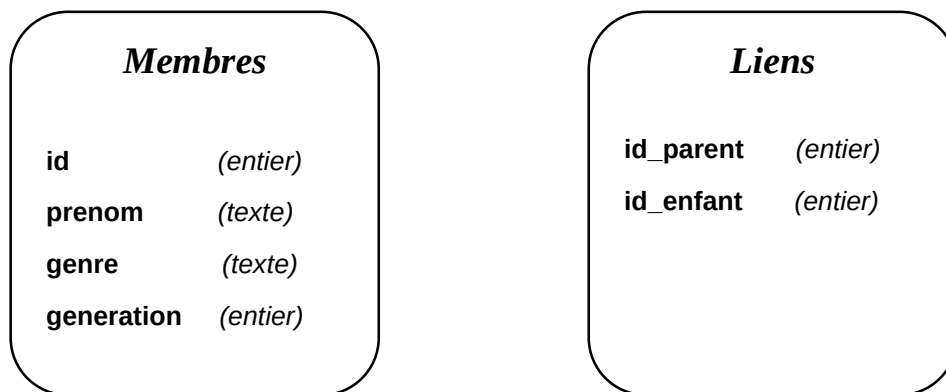
Pour plus de clarté, nous prendrons l'arbre généalogique suivant comme exemple dans toute cette partie :



Cet arbre généalogique est composé de plusieurs générations. Le plus grand ancêtre est 'Mohamed', il appartient à la génération **0**. Le plus grand ancêtre est appelé : **racine** de l'arbre. Ce dernier possède trois enfants : 'Ahmed', 'Maryem' et 'Khalid'. Ces trois enfants appartiennent à la génération **1**. 'Khalid' n'a pas d'enfant, 'Ahmed' a un enfant, et 'Maryem' en a deux. Les enfants de 'Ahmed' et de 'Maryem' appartiennent à la génération **2**, ... etc. Le nombre total de générations dans l'arbre est appelé : **hauteur** de l'arbre. On peut calculer la hauteur de l'arbre en ajoutant le nombre **1** à la plus grande génération dans l'arbre.

Exemple : La hauteur de l'arbre de l'exemple précédent est **4**.

Dans ce qui suit, on propose de représenter l'arbre généalogique par une base de données composée de deux tables : la table : **membres** et la table : **liens**.



a- Structure de la table Membres :

Membres (id , prenom , genre , generation)

La table **Membres** contient tous les membres de l'arbre généalogique. Elle est composée de **quatre** champs :

- ✓ Le champ **id** contient un entier positif unique, qui permet d'identifier chaque membre dans cette table ;
- ✓ Le champ **prenom** contient le prénom d'un membre de l'arbre généalogique ;
- ✓ Le champ **genre** contient le caractère '**M**' pour les membres masculins, ou '**F**' pour les membres féminins ;
- ✓ Le champ **generation** contient un entier positif qui représente le niveau du membre dans l'arbre généalogique.

b- La structure de la table Liens :

Liens (id_parent , id_enfant)

La table **Liens** contient tous les liens de parenté *parent-enfant* entre les membres de l'arbre généalogique. Elle est composée de deux champs :

- ✓ Le champ **id_parent** contient les identifiants des membres parents ;
- ✓ Le champ **id_enfant** contient les identifiants des membres enfants.

Épreuve d'Informatique – Session 2017 – Filière MP/ PSI/ TSI

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- ✓ L'épreuve se compose de trois parties indépendantes.
- ✓ Toutes les instructions et les fonctions demandées seront écrites en Python.
- ✓ Les questions non traitées peuvent être admises pour aborder les questions ultérieures.
- ✓ Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

_ * _ * _ * _ * _ * _ * _ * _ * _ * _

Partie I : Base de données et langage SQL

La compression des fichiers est une pratique aussi courante qu'indispensable en informatique. Elle consiste à stocker les données dans un nouveau format moins encombrant que l'original. Ce gain de place induit d'autres bénéfices dont le principal est l'accélération des transferts entre ordinateurs. Faire circuler moins de bits sur les réseaux diminue le temps de connexion, encombre moins les lignes de communication et limite les dépenses télématiques. La décompression rétablit le fichier dans son état initial.

De nombreux algorithmes de compressions existent, chacun ayant sa particularité et surtout un type de données cible. Car toutes les données ne se compressent pas de la même manière. Un algorithme de compression de texte travaillera sur les répétitions du nombre de caractères ou de parties de phrases. Un algorithme de compression d'images travaillera quant à lui sur d'autres domaines comme la différence entre un pixel et un autre. On imagine cependant mal le second algorithme en train de compresser un texte.

Néanmoins, tous les algorithmes de compression ont un point commun : leur objectif est de récupérer les données initiales partiellement, voire totalement. La décompression consiste à rétablir les données d'origine à l'aide du fichier compressé. Elle consiste souvent à appliquer l'algorithme de compression en sens inverse.

Types de compressions :

Il y a deux types majeurs de compressions : la compression sans perte et la compression avec perte.

a- La compression sans perte : Une compression est dite sans perte si les données après décompression sont identiques aux données originelles.

Ces compression se basent toutes sur le même principe : la répétition d'une donnée est une répétition de trop. L'objectif va être de supprimer le maximum de répétition pour obtenir une compression plus importante tout en étant capable de retrouver les répétitions retirées.

En somme, ces compressions écrivent exactement les mêmes données mais de façon plus concise.

Elles sont appliquées à tous types de données et les formats compressés sont très nombreux. Pour ne citer que les plus connus, nous retrouvons les formats : zip, rar, 7z, bz2, gz, ...

Les algorithmes, moins connus du grand public, sont aussi très nombreux. Nous retrouvons par exemple le codage de Huffman, les codages de Lempel-Ziv, ...

b- La compression avec perte : Une compression est dite avec perte si les données après décompression sont différentes des données originelles.

Elles sont appliquées à des données perceptibles, c'est-à-dire à des images, des sons ou des vidéos. Le principe va consister à supprimer les informations là où les sens de la vue et de l'ouïe ne les perçoivent que très peu. Par exemple, l'œil humain ne distingue que très peu les zones de contraste. Aussi, nous pouvons retirer des détails à ces zones sans trop impacter sur la qualité de l'image. Le nom de format représente directement le type de compression employé : jpeg, mpeg, mp3, divx, ...

Taux de compression :

Le taux de compression est une mesure de la performance d'un algorithme de compression. Il est exprimé en pourcentage. Le taux de compression est le gain en volume rapporté au volume initial des données. Plus le taux de compression est élevé, plus la taille du fichier compressé résultant est faible. La formule correspondante s'écrit :

$$\text{Taux} = \left(1 - \frac{\text{taille du fichier compressé}}{\text{taille initiale du fichier}}\right) * 100$$

On dispose d'une base de données de chemin absolu : "**C:\BDcompressions.sql**", composée de trois tables : la table **Algorithme**, la table **Fichier** et la table **Compression**, dont les structures sont les suivantes :

➤ **Algorithme** (format (texte), **type** (entier))

La table **Algorithme** contient différents formats d'algorithmes de compressions. Le champ **format** est la clé primaire. Le champ **type** contient la valeur **1**, si l'algorithme de compression est du type avec perte de données, ou **0** sinon.

Exemples de données de la table Algorithme :

format	type
7z	0
bz2	0
gz	0
jpeg	1
rar	0
zip	0
mp3	1
huffman	0
...	...

➤ **Fichier** (numeroF (entier), **nomF** (texte), **tailleF** (entier))

La table **Fichier** contient des fichiers de différents types : texte, image, son, vidéo Le champ **numeroF** est la clé primaire. Le champ **nomF** contient le nom complet de chaque fichier. Le champ **tailleF** contient la taille originale du fichier, exprimée en Octet.

Exemples de données de la table Fichier :

numeroF	nomF	tailleF
1	Cours Python.pdf	6 420 768
2	Nature.bmp	5 760 054
3	Langage SQL.doc	345 600
4	Chanson.wav	47 185 920
...	...	...

➤ **Compression** (**numeroF** (entier), **format** (texte), **tailleC** (entier))

La table **Compression** contient les compressions de certains fichiers de la table Fichier. Les champs **numeroF** et **format** sont des clés étrangères. Le champ **tailleC** contient la taille du fichier compressé, exprimée en octet. Un fichier de la table Fichiers peut être compressé plusieurs fois, par différents algorithmes de compressions.

Exemples de données de la table Compression :

numeroF	format	tailleC
2	jpeg	291 441
2	png	2 622 545
2	zip	2 160 937
1	cab	5 768 938
1	zip	5 818 538
4	mp3	11 364
3	zip	145 366
3	7z	133 396
...	...	...

I. 1- Écrire, en algèbre relationnelle, une requête qui donne pour résultat : Les noms des fichiers dont la taille originale est comprise entre 1 Kilo-octet et 1 Méga-octet.

I. 2- Écrire une requête SQL, qui donne pour résultat : Les noms et les tailles des fichiers textes, dont le nom se termine par : .doc ou .docx, triés dans l'ordre alphabétique des noms des fichiers.

I. 3- Écrire une requête SQL qui donne pour résultat : Les noms des fichiers compressés, les formats de compression, les types de compression, et le taux de compression de chaque fichier, dont le taux de compression dépasse 40%, triés dans l'ordre des numéros des fichiers.

I. 4- Écrire une requête SQL qui donne pour résultat : Les algorithmes sans perte de données et le compte des fichiers compressés par chacun de ces algorithmes, dont ce compte est égal à 3, 5 ou 8.

I. 5- Écrire une requête SQL qui donne pour résultat : Les 3 premiers grands taux de compressions, triés dans l'ordre croissant.

I. 6- Écrire un programme Python qui saisi un format de compression, et qui affiche **le taux moyen** des taux des compressions de ce format, si le format saisi existe dans la base de données. Si le format saisi n'existe pas, le programme doit afficher le message : « *Le format saisi n'existe pas dans la BD* ».

On suppose que le module **sqlite3** est importé. Ce module contient les fonctions et les méthodes permettant de manipuler les bases de données :

Connexion à une base de données : connect ()	Exécuter une requête : execute ()
Créer un curseur : cursor ()	Récupérer le résultat d'une requête : fetchone () , fetchall ()

Épreuve d'informatique – Session 2015 – Filière MP/ PSI/ TSI

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- L'épreuve se compose de deux exercices et d'un problème.
- Toutes les instructions et les fonctions demandées seront écrites en langage Python.
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures.
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

Exercice 1. Thème : base de données

Nous allons nous intéresser à la gestion d'une compagnie de trains de marchandises en nous limitant à la gestion des trains. La base de données permettant la gestion des différents trains de cette compagnie sera constituée de 3 tables : *TRAIN*, *TRAJET*, *TYPE*.

La description de ces tables est la suivante :

<i>TRAIN</i>	<i>TRAJET</i>	<i>TYPE</i>
immatriculation : entier gare_attache : texte id : entier	num_trajet : entier immatriculation : entier ville_dep : texte ville_arr : texte heure_dep : date	id : entier nom : texte nb_place : entier

Une clé étrangère *id* est placée sur la table *TRAIN* et mise en relation avec la clé primaire *id* de la table *TYPE*.

Une clé étrangère *immatriculation* est placée sur la table *TRAJET* et mise en relation avec la clé primaire *immatriculation* de la table *TRAIN*.

Une clé primaire *num_trajet* est placée sur la table *TRAJET*.

Autre représentation :

TRAIN (immatriculation, gare_attache, *id*)

Exemple : (2034, "Fez", 2)

TRAJET (num_trajet, immatriculation, ville_dep, ville_arr, heure_dep, heure_arr)

Exemple : (789, 2034, "Rabat", "Casablanca", "09h00", "10h20")

TYPE (id, nom, nb_place)

Exemple : (2, "TNR", 440)

Dans la représentation précédente les clefs principales sont en caractères gras et les clefs secondaires en italique et soulignés.

Vous pouvez utiliser le système de base de données que vous souhaitez. Cette base de données pourra être de type **SQLite** ou **MySQL** ou **PostgreSQL** etc.

Si vous utilisez une base de données **SQLite**, le fichier « *gestion_trains.sqlite* » représentant cette base de données se trouvera dans le même répertoire que vos programmes **Python** (sous C:\CNC\)

de cette compagnie ferroviaire.
Dans cet exercice la base de données est déjà créée : tables avec leurs contenus.

Question 1 :

Écrire une requête **SQL** donnant comme résultat l'heure de départ, l'heure d'arrivée et l'identifiant de tous les trains au départ de la ville de Fez.

Question 2 :

Modifier la requête précédente afin d'afficher, en plus des informations demandées, la gare d'attache de tous les trains toujours au départ de la ville de Fez. Attention on ne veut pas de doublon.

Question 3 :

Écrire une requête **SQL** donnant comme résultat le nombre de trajets au départ de Rabat par type de train.

Question 4 :

Écrire une requête **SQL** donnant le nom et la gare de départ de tous les trains dont l'heure de départ est comprise entre 7h et 13h.

Question 5 :

Écrire en **Python** le programme complet qui affichera le résultat de l'une des requêtes correspondant aux quatre premières questions. Vous pouvez utiliser le système de base de données que vous voulez, la création et le contenu des trois tables sont créés par défaut.